

Comparison Sorting

Quicksort usually has a running time of $n \times \log(n)$, but is there an algorithm that can sort even faster? In general, this is not possible. Most sorting algorithms are comparison sorts, i.e. they sort a list just by comparing the elements to one another. A comparison sort algorithm cannot beat $n \times \log(n)$ (worst-case) running time, since $n \times \log(n)$ represents the minimum number of comparisons needed to know where to place each element. For more details, you can see [these notes](#) (PDF).

Alternative Sorting

Another sorting method, the counting sort, does not require comparison. Instead, you create an integer array whose index range covers the entire range of values in your array to sort. Each time a value occurs in the original array, you increment the counter at that index. At the end, run through your counting array, printing the value of each non-zero valued index that number of times.

Example

arr = [1, 1, 3, 2, 1]

All of the values are in the range [0...3], so create an array of zeros, **result** = [0, 0, 0, 0]. The results of each iteration follow:

[Change Theme](#)

Language

Java 15



```

1  import java.io.*;
2  import java.math.*;
3  import java.security.*;
4  import java.text.*;
5  import java.util.*;
6  import java.util.concurrent.*;
7  import java.util.function.*;
8  import java.util.regex.*;
9  import java.util.stream.*;
10 import static java.util.stream.Collectors.joiner;
11 import static java.util.stream.Collectors.toList;
12
13 class Result {
14     public static List<Integer> countingSort(List<Integer> arr) {
15         List<Integer> frequency = new ArrayList<>();
16         int n=100;
17         for(int i=0;i<n;i++){
18             frequency.add(0);
19         }
20         for(int i=0;i<arr.size();i++){
21             int value = arr.get(i);
22             int CurrentCount = frequency.get(value);
23             frequency.set(value, CurrentCount+1);
24         }
25         return frequency;
26     }
27 }
28

```

Line: 16 Col: 19